

# Notion Study 섹션 원고

Portfolio 노션의 **STUDY** 섹션에 서브페이지로 정리하기 좋은 형태입니다.

현재 노션 **논문** 페이지의 흐름인 **Transformer** → **GPT-1** → **BERT** → **T5** → **UL2**를 중심축으로 두고, Vision Transformer / Prompt-to-Prompt / RAG까지 확장 학습 기록으로 묶었습니다.

## Study 카드 요약

| 제목                                                                                | 분야                     | 연도   | 읽은 포인트                                                      | 포트폴리오용 한 줄                                                         |
|-----------------------------------------------------------------------------------|------------------------|------|-------------------------------------------------------------|--------------------------------------------------------------------|
| Attention Is All You Need                                                         | Transformer / NLP      | 2017 | Self-Attention, Q/K/V, Encoder-Decoder, Positional Encoding | RNN 없이 Attention만으로 sequence modeling을 처리하는 Transformer의 기본 구조를 정리 |
| Improving Language Understanding by Generative Pre-Training                       | GPT / Generative LM    | 2018 | Decoder-only, Generative Pre-training, Fine-tuning          | 생성형 사전학습이 downstream task 성능으로 이어지는 흐름을 GPT-1 기준으로 이해              |
| BERT: Pre-training of Deep Bidirectional Transformers                             | NLP Understanding      | 2019 | Encoder-only, MLM, NSP, Token/Segment/Position Embedding    | 양방향 문맥 이해를 위해 Encoder-only 구조와 MLM 사전학습이 왜 필요한지 정리                 |
| Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer | T5 / Transfer Learning | 2019 | Text-to-Text, task unification, transfer learning           | 모든 NLP 문제를 text-to-text 형식으로 통일하는 관점을 학습                           |
| UL2: Unifying Language Learning Paradigms                                         | Language Modeling      | 2022 | Denoising objectives, mixture-of-denoisers                  | 이해형·생성형 학습 목표를 하나의 언어 모델 학습 패러다임으로 통합하는 방향을 정리                     |
| An Image is Worth 16x16 Words                                                     | Vision Transformer     | 2021 | Patch Embedding, CLS token, Transformer Encoder             | 이미지를 16x16 패치 토큰으로 바꿔 Transformer Encoder에 넣는 ViT 구조를 정리           |

| 제목                                                               | 분야                        | 연도   | 읽은 포인트                                         | 포트폴리오용 한 줄                                                  |
|------------------------------------------------------------------|---------------------------|------|------------------------------------------------|-------------------------------------------------------------|
| Prompt-to-Prompt Image Editing with Cross Attention Control      | Diffusion / Image Editing | 2022 | Cross-Attention Control, prompt-based editing  | 마스크 없이 Cross-Attention을 제어해 텍스트만으로 이미지 구조를 보존하며 편집하는 방법을 정리 |
| Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks | RAG                       | 2020 | Retriever + Generator, RAG-Sequence, RAG-Token | 지식을 파라미터에만 저장하지 않고 검색 문서를 조건으로 생성하는 RAG 구조를 정리              |

## Study 섹션 소개 문안

논문을 읽을 때 단순히 모델 이름을 외우기보다, “이 논문은 이전 방식의 어떤 병목을 해결했는가?”를 중심으로 정리했습니다. Transformer 계열 논문은 **구조의 변화**를, RAG와 Prompt-to-Prompt는 **모델을 서비스로 쓸 때 필요한 제어 방식**을 중심으로 읽었습니다.

특히 포트폴리오 프로젝트와 연결되는 지점은 두 가지입니다.

- RAG 논문 학습 → PyMate와 WorkFlow Agent에서 검색 품질, 근거 기반 생성, hallucination 제어 기준으로 연결
- Transformer / BERT / ViT 학습 → Attention, embedding, encoder 구조를 코드와 다이어그램으로 직접 해석하는 기반으로 연결

## 1. Attention Is All You Need (2017)

### 핵심 질문

RNN이나 CNN 없이 Attention만으로 문장 안의 단어 관계를 충분히 학습할 수 있을까?

### 내가 정리한 핵심

Transformer는 문장을 순차적으로 처리하지 않고, Self-Attention으로 모든 토큰 간 관계를 한 번에 계산합니다. 노션 메모에서는 Q(Query), K(Key), V(Value)를 각각 “찾고 싶은 정보”, “각 단어의 특징”, “실제 의미 정보”로 비유해 정리했습니다.

### 주요 개념

- Self-Attention: 토큰이 문장 안의 다른 토큰을 얼마나 참고할지 계산
- Q/K/V: attention score와 실제 정보 집계를 분리하는 구조
- Multi-Head Attention: 여러 관점에서 단어 관계를 동시에 학습
- Positional Encoding: 순서를 직접 처리하지 않는 Attention에 위치 정보를 추가
- Encoder-Decoder: 입력 문맥 이해와 출력 생성 역할을 분리

### 학습 메모

RNN은 순서대로 읽기 때문에 긴 문장에서 병렬화가 어렵고 long-range dependency가 약해집니다. Transformer는 이 병목을 Self-Attention으로 해결했고, 이후 BERT, GPT, T5, ViT까지 확장되는 공통 구조가 되었습니다.

## 프로젝트 연결

PyMate와 WorkFlow Agent에서 RAG pipeline을 설계할 때, 검색된 chunk를 단순히 이어 붙이는 것이 아니라 "어떤 토큰/문서가 답변에 더 중요한가"를 attention 관점으로 이해하는 데 기반이 되었습니다.

## 2. Improving Language Understanding by Generative Pre-Training (GPT-1, 2018)

### 핵심 질문

대규모 unlabeled text로 먼저 언어를 학습한 뒤, 적은 supervised data로 다양한 NLP task에 적응할 수 있을까?

### 내가 정리한 핵심

GPT-1은 Transformer Decoder를 사용해 왼쪽에서 오른쪽으로 다음 토큰을 예측하는 방식으로 사전학습합니다. 이후 task-specific input format과 fine-tuning을 통해 분류, 질의응답, 자연어 추론 같은 downstream task에 적용합니다.

### 주요 개념

- Decoder-only Transformer
- Generative Pre-training
- Fine-tuning
- Left-to-right language modeling
- Task-specific input transformation

### 학습 메모

BERT가 문장을 "이해"하기 위해 Encoder를 선택했다면, GPT는 다음 단어를 "생성"하기 위해 Decoder를 선택했습니다. 이 차이가 이후 생성형 AI와 이해형 NLP 모델의 기본 분기점이 됩니다.

## 프로젝트 연결

WorkFlow Agent에서 sLLM이 JSON 판단 결과를 생성하도록 만들 때, 생성형 모델은 구조화된 출력에 강제 조건을 주지 않으면 형식이 흔들릴 수 있다는 점을 체감했습니다. GPT 계열의 생성 방식 이해는 JSON 유효성 검증과 fallback parser를 설계하는 배경이 되었습니다.

## 3. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (2019)

### 핵심 질문

문장을 한 방향으로만 읽는 모델은 앞뒤 문맥을 동시에 이해하기 어렵다. 양방향 문맥을 학습하려면 어떤 사전 학습 방식이 필요할까?

## 내가 정리한 핵심

BERT는 Transformer의 Encoder만 사용해 문장 전체를 동시에 봅니다. 노션 메모에서는 "은행"처럼 앞뒤 문맥을 함께 봐야 의미가 결정되는 예시를 통해, 양방향 이해가 왜 필요한지 정리했습니다.

## 주요 개념

- Encoder-only Transformer
- Masked Language Model (MLM)
- Next Sentence Prediction (NSP)
- Token Embedding + Segment Embedding + Position Embedding
- Fine-tuning for downstream tasks

## 학습 메모

BERT의 MLM은 일부 토큰을 가리고 주변 문맥으로 맞히게 합니다. 마스킹된 토큰 중 80%는 [MASK], 10%는 random token, 10%는 그대로 유지해 모델이 [MASK]에만 과적합하지 않도록 설계합니다.

BERT 입력은 세 가지 임베딩의 합으로 구성됩니다.

Input Embedding = Token Embedding + Segment Embedding + Position Embedding

## 프로젝트 연결

WorkFlow Agent의 Intent 분류와 PyMate의 검색 질의 이해를 볼 때, "생성"보다 "분류·이해"가 중요한 구간에서는 Encoder 계열 모델의 장점이 더 크다는 판단 기준을 얻었습니다.

---

## 4. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5, 2019)

### 핵심 질문

번역, 요약, 분류, 질의응답처럼 서로 다른 NLP task를 하나의 형식으로 통일할 수 있을까?

## 내가 정리한 핵심

T5는 모든 NLP 문제를 **text-to-text** 형식으로 바꿉니다. 입력도 텍스트, 출력도 텍스트로 통일함으로써 task 별 head나 구조를 크게 바꾸지 않고 transfer learning을 실험할 수 있게 했습니다.

## 주요 개념

- Text-to-Text framework
- Encoder-Decoder Transformer
- Task prefix
- Transfer learning
- C4 dataset

## 학습 메모

T5의 핵심은 모델 구조 자체보다 “문제를 어떤 인터페이스로 통일할 것인가”에 있습니다. 분류도 텍스트 생성으로 보고, 요약도 텍스트 생성으로 보면 모델 입장에서는 모든 문제가 같은 입출력 형식을 가집니다.

### 프로젝트 연결

PyMate의 스튜디오 기능에서 **요약**, **단계별 설명**, **플래시카드**, **비교표**, **다른 예시**를 모두 같은 LLM 호출 패턴으로 정리할 수 있었던 배경이 됩니다. 기능은 달라도 입력 지시와 출력 포맷을 설계하면 하나의 생성 파이프라인으로 다룰 수 있습니다.

---

## 5. UL2: Unifying Language Learning Paradigms (2022)

### 핵심 질문

언어모델 학습 목표가 causal LM, masked LM, sequence-to-sequence denoising으로 나뉘어 있는데, 이를 하나로 통합할 수 있을까?

### 내가 정리한 핵심

UL2는 서로 다른 언어모델 학습 패러다임을 mixture-of-denoisers로 통합합니다. 짧은 span을 복원하는 objective, 긴 span을 복원하는 objective, prefix 기반 생성 objective를 함께 사용해 이해와 생성 능력을 동시에 강화하려는 접근입니다.

### 주요 개념

- Mixture-of-Denoisers
- R-denoising
- S-denoising
- X-denoising
- Mode switching

### 학습 메모

BERT는 이해, GPT는 생성, T5는 text-to-text 통합에 강점이 있다면, UL2는 학습 objective 자체를 섞어 다양한 언어 사용 상황을 하나의 모델이 다루도록 설계합니다.

### 프로젝트 연결

Workflow Agent처럼 하나의 시스템 안에서 문서 요약, 규정 판단, 일반 질의, 계획 분해가 섞이는 경우, 단일 task에 특화된 모델보다 다양한 objective를 건딜 수 있는 모델 설계가 중요하다는 관점으로 이어집니다.

---

## 6. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT, 2021)

### 핵심 질문

이미지도 단어처럼 토큰화하면, CNN 없이 Transformer만으로 이미지 분류를 할 수 있을까?

### 내가 정리한 핵심

ViT는 이미지를 16x16 패치로 자르고, 각 패치를 하나의 token처럼 Transformer Encoder에 넣습니다. 노션 메모에서는 224x224 이미지를 16x16 패치로 자르면 14x14, 총 196개 패치가 생긴다는 흐름으로 정리했습니다.

## 주요 개념

- Patch Embedding
- Linear Projection
- Positional Embedding
- [CLS] token
- Transformer Encoder
- MLP Head

## 학습 메모

ViT의 구조는 BERT와 매우 유사합니다. 단어 대신 이미지 패치가 들어가고, 문장 대표 [CLS] token처럼 이미지 전체를 대표하는 [CLS] token을 사용합니다.

```
Image → 16x16 patches → Patch Embedding
      → Position Embedding + CLS
      → Transformer Encoder
      → MLP Head
      → Class prediction
```

## 프로젝트 연결

Transformer가 텍스트에만 쓰이는 구조가 아니라는 점을 이해했습니다. 이후 이미지 생성/편집 논문인 Prompt-to-Prompt를 읽을 때, 이미지 내부의 공간 정보와 attention map을 연결해서 이해하는 기반이 되었습니다.

# 7. Prompt-to-Prompt Image Editing with Cross Attention Control (2022)

## 핵심 질문

기존 이미지의 구조는 유지하면서, 마스크를 직접 그리지 않고 텍스트 프롬프트만으로 원하는 부분을 편집할 수 있을까?

## 내가 정리한 핵심

텍스트 기반 이미지 생성 모델은 프롬프트가 조금만 바뀌어도 이미지 전체가 바뀌기 쉽습니다. Prompt-to-Prompt는 Cross-Attention layer가 프롬프트의 단어와 이미지의 공간 구조를 연결한다는 점에 주목해, attention map을 제어함으로써 구조를 보존한 편집을 가능하게 합니다.

## 주요 개념

- Text-conditioned diffusion model
- Cross-Attention Control
- Localized Editing
- Global Editing
- Attention re-weighting

### 노션 메모 정리

기존 마스크 기반 편집은 사용자가 직접 수정 영역을 그려야 하고, 마스크 내부의 구조와 질감이 무너질 수 있습니다. 반면 Prompt-to-Prompt는 단어/토큰 단위로 의미를 바꿉니다.

| 비교 항목  | 마스크 기반 편집 | Prompt-to-Prompt     |
|--------|-----------|----------------------|
| 제어 방식  | 공간 기반     | 의미 기반                |
| 수정 단위  | 픽셀 영역     | 단어 / 토큰              |
| 사용자 개입 | 큼         | 작음                   |
| 구조 보존  | 불안정       | Cross-Attention으로 보존 |

### 프로젝트 연결

이 논문은 “모델의 출력을 제어하려면 어디를 잡아야 하는가”라는 관점을 줍니다. WorkFlow Agent에서도 LLM 답변 전체를 사후 수정하기보다, 검색 근거·출력 schema·confidence 같은 제어 지점을 앞단에 설계해야 한다는 생각과 연결됩니다.

## 8. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG, 2020)

### 핵심 질문

모델이 모든 지식을 파라미터에 암기하는 대신, 필요할 때 외부 문서를 검색해 그 근거를 보고 생성하게 만들 수 있을까?

### 내가 정리한 핵심

RAG는 Retriever와 Generator를 결합합니다. 질문이 들어오면 외부 지식 문서에서 관련 문서를 검색하고, 생성 모델은 그 문서를 조건으로 답변을 생성합니다. 노션 메모에서는 “모델이 모든 지식을 파라미터에만 저장하려 하지 말고, 필요할 때 외부 문서를 검색해서 그 근거를 보면서 생성하자”는 문장으로 정리했습니다.

### 주요 개념

- Parametric memory vs Non-parametric memory
- Dense Passage Retrieval (DPR)
- Retriever + Generator
- RAG-Sequence
- RAG-Token
- Top-K marginalization

### 노션 메모 정리

기존 생성 모델은 지식을 파라미터에 저장하기 때문에 최신 지식 업데이트가 어렵고, 근거 제시가 약하며, hallucination에 취약합니다. RAG는 외부 문서를 동적으로 검색하고 그 문서를 조건으로 답변을 생성해 이 문제를 완화합니다.

RAG의 두 가지 변형은 다음처럼 이해했습니다.

| 방식           | 직관                      | 장점                | 한계                           |
|--------------|-------------------------|-------------------|------------------------------|
| RAG-Sequence | 하나의 문서를 고르고 전체 답변을 생성   | 단순하고 구현이 상대적으로 쉬움 | 답의 부분마다 다른 문서가 필요한 경우 유연성 낮음 |
| RAG-Token    | 토큰마다 참고 문서를 다르게 섞을 수 있음 | 여러 근거를 조합하는데 유리   | 계산량과 근사 처리가 복잡               |

한계와 주의점

- 검색이 틀리면 생성도 틀립니다.
- 문서를 줘도 hallucination이 완전히 사라지는 것은 아닙니다.
- 인덱스 구축, 검색 latency, 최신 문서 반영 같은 운영 복잡도가 생깁니다.

프로젝트 연결

PyMate에서는 RAGAS로 검색 품질을 측정하며 병목이 LLM이 아니라 embedding 검색 품질에 있음을 확인했습니다. Workflow Agent에서는 규정 판단 Agent에 HyDE + BM25 + Vector + RRF + Reranker를 적용해, 단일 검색이 놓치는 규정 교차 판단 문제를 줄였습니다.

Portfolio Study 페이지 구성 추천

첫 화면

AI Paper Study

Transformer 계열 모델이 어떻게 이해형(BERT), 생성형(GPT), 통합형(T5/UL2), 비전 모델(ViT), 검색 결합 모델(RAG)로 확장되는지 논문 단위로 정리했습니다.

읽는 기준:

1. 이 논문이 해결한 이전 방식의 병목
2. 핵심 구조와 학습 목표
3. 내 프로젝트에 연결되는 판단 기준

추천 하위 페이지

- Transformer Family
  - Attention Is All You Need
  - GPT-1
  - BERT
  - T5
  - UL2
- Vision & Diffusion
  - Vision Transformer
  - Prompt-to-Prompt
- RAG & Agent
  - Retrieval-Augmented Generation



## 포트폴리오에 보여줄 핵심 문장

논문을 단순 요약으로 끝내지 않고, 프로젝트 설계 기준으로 연결했습니다. RAG 논문은 PyMate와 WorkFlow Agent의 검색 품질 평가 기준으로, BERT/GPT/T5 흐름은 이해형·생성형·통합형 모델을 어떤 작업에 써야 하는지 판단하는 기준으로 정리했습니다.